SYSTEM ANALYSIS DOCUMENT (SAD)

Coordina AI

Autonomous AI Teammate & Workflow Orchestrator

By Team BU ZHI DAO

UM HACKATHON 2026

Table of Content

1. Introduction **3**
1.1. Purpose 3
1.2. Background 3
1.3. Target Stakeholder 4
2. System Architecture & Design **4**
2.1. High Level Architecture 4
2.1.1. Overview 4
2.1.2. LLM as Service Layer 5
2.1.3. Dependency Diagram 6
2.1.4. Sequence Diagram 7
2.2. Technological Stack 8
2.3. Key Data Flows 8
2.3.1. Data Flow Diagram (DFD) 8
2.3.2. Normalized Database Schema 8
3. Functional Requirements & Scope **9**
3.1. Minimum Viable Product 9
3.2. Non-Functional Requirements (NFRs) 9
3.3. Out of Scope / Future Enhancements 10
4. Monitor, Evaluation, Assumptions & Dependencies **11**
4.1. Technical Evaluation 11
4.1.1. Grayscale Rollout and Comparative Testing Strategy 11
4.1.2. Emergency Rollback (ER) & Golden Release 11
4.1.3. Priority Matrix 11
4.2. Monitoring 12
4.3. Assumptions 13
4.4. External Dependencies 13
5. Project Management & Team Contribution **14**
5.1. Project Timeline 14
5.2. Team Members Role 15
5.3. Recommendations 16

1. Introduction

Coordina AI is a GLM-powered workflow orchestration system that addresses fragmented student collaboration by converting unstructured project inputs into structured, traceable execution workflows. It acts as an autonomous AI teammate for project analysis, planning, coordination, risk monitoring, and submission readiness.

1.1. Purpose

This document defines the implemented technical scope, architecture and system behavior or Coordina AI. It serves as the engineering and quality assurance reference to align implementation, validation and demonstration.

Key Elements Covered

- **Architecture:** Modular FastAPI backend with orchestrator, specialized agents, parsers, Redis state memory, PostgreSQL persistence, and GLM service integration.
- **Data Flow:** Begin with input ingestion, followed by parser normalization, after which the agents perform reasoning using the GLM model. The system then updates its internal state, generates decisions or events, and finally delivers responses through the frontend or APIs.
- **Model Process:** End-to-end staged workflow consisting of analysis, planning, coordination, risk checking and recovery, and submission readiness evaluation.
- **Role of Reference:** Shared technical reference for developers and testers, outlining APIs, system boundaries, failure handling mechanisms, and the overall development roadmap.

1.2. Background

Student teams currently rely on fragmented tools and manual coordination. Existing platforms store project artifacts but do not provide stateful reasoning across project stages, causing ambiguity and late interventions.

Previous Version

Manual collaboration baseline using generic tools (documents / chats / task boards) with no centralized reasoning engine, no cross-stage memory, and no autonomous risk interventions.

Major Architecture Changes Introduced

- GLM reasoning service (ilmu-glm-5.1) integrated as core orchestration intelligence.
- Role-specialized agents for stage-based execution.
- Redis-backed shared project state and decision logging.
- Edge-case modules for ambiguity, missing data, inactivity, and deadline recovery.
- Workflow endpoints for stage-by-stage and full-pipeline execution.

1.3. Target Stakeholder

Stakeholder	Roles	Expectations
Team Leader	Manage project execution	Automated planning, task allocation, monitoring
Team Members	Collaborate on project	Clear tasks, structured workflow, timely completion
Development Team	Build and maintain system	Modular architecture, clear APIs
Quality Assurance Team	Validate System	Defined flows, edge case handling

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview

Type	Details
System	Web-based Application
Architecture	Client-server, modular service-oriented backend

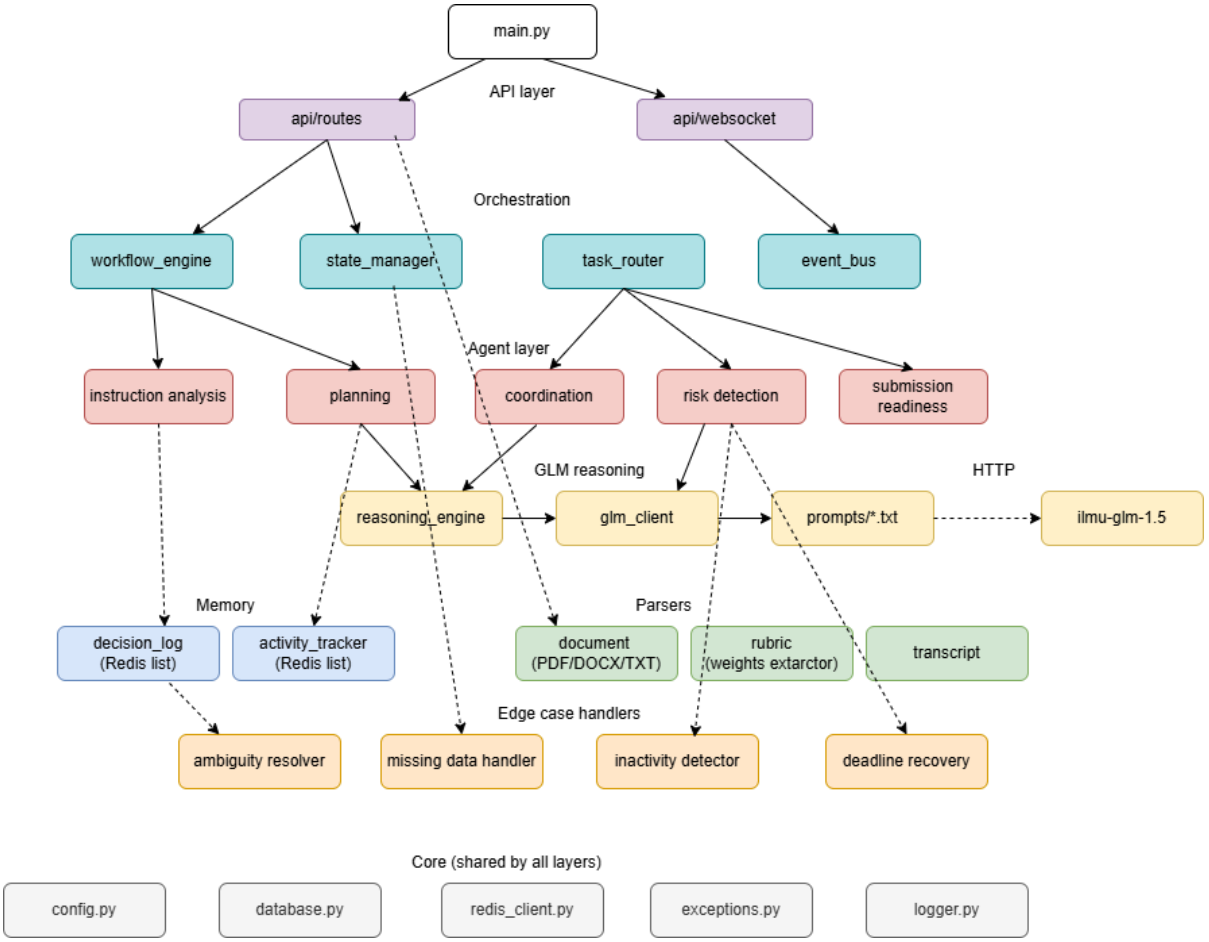
The system is designed as a web-based application consisting of a frontend interface and a backend orchestration API. It follows a client-server architecture with a modular, service-oriented backend that supports scalability and maintainability. The core components include

the frontend for user interaction, FastAPI routes for handling requests, a central Workflow Engine to manage execution logic, specialized agents for different tasks, a GLM-based reasoning engine for AI-driven decision-making, Redis for in-memory state management, and PostgreSQL for persistent data storage.

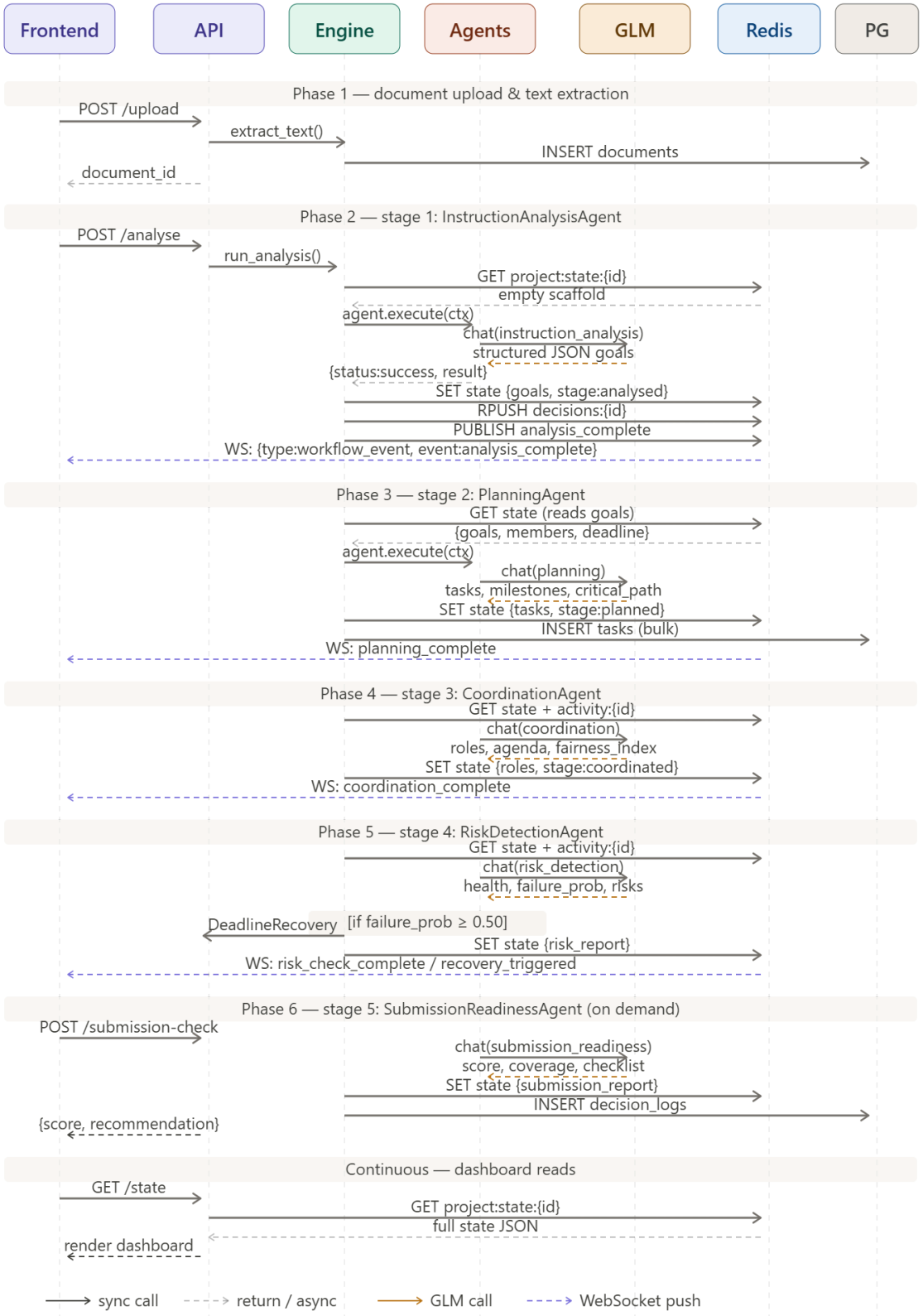
2.1.2. LLM as Service Layer

The system uses predefined prompt templates for each agent, which are dynamically loaded and enriched with context in JSON format before being processed. Requests are sent asynchronously to the GLM API, ensuring efficient handling of multiple operations. The responses returned by the model are parsed into structured JSON and then propagated to the appropriate next stage or system state for further processing. To ensure reliability, API-level retry mechanisms are implemented to handle transient HTTP failures. Context size is currently managed through bounded excerpts and truncation in selected pathways, while a more advanced semantic chunking and relevance-based retrieval pipeline is planned for future enhancement.

2.1.3. Dependency Diagram



2.1.4. Sequence Diagram

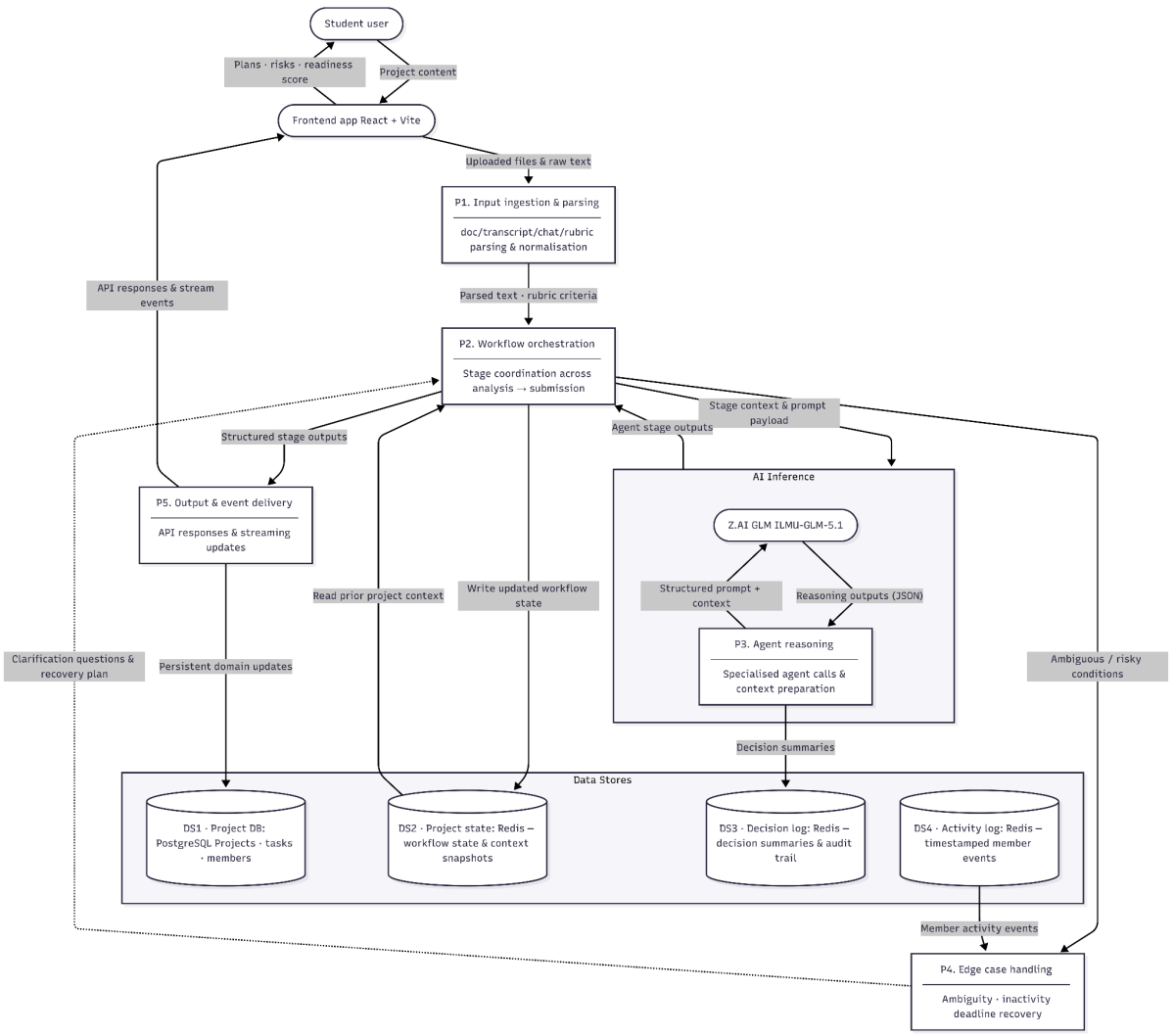


2.2. Technological Stack

- Frontend: React + Vite
- Backend: FastAPI (Python async)
- Database: PostgreSQL
- State Memory: Redis
- AI Service: Z.AI API with model ILMU-GLM-5.1
- Testing: pytest

2.3. Key Data Flows

2.3.1. Data Flow Diagram (DFD)



2.3.2. Normalized Database Schema

SYSTEM ANALYSIS DOCUMENTATION (ER-DIAGRAM).png

3. Functional Requirements & Scope

3.1. Minimum Viable Product

Features	Description
Requirement Analysis and Goal Extraction	Users submit project briefs, rubrics, meeting transcripts, or chat logs, and the system extracts structured goals, deliverables, grading priorities, and ambiguity indicators for downstream planning.
Workflow Planning and Milestone Generation	Based on extracted goals, the system generates a project execution plan that includes tasks, milestones, and dependency-aware sequencing to guide team delivery.
Team Coordination and Role Assignment	The system assigns roles and distributes responsibilities using available member and task context, while generating a meeting agenda to improve team alignment and continuity.
Risk Monitoring and Recovery Planning	The system detects inactivity and deadline risk, and when risk thresholds are exceeded, it generates a recovery plan with timeline compression and task reprioritization recommendations.
Submission Readiness Validation	The system evaluates completed deliverables and uploaded artifacts against rubric criteria, then produces a readiness score, coverage summary, and final checklist guidance before submission.

3.2. Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Scalability	The system must support concurrent workflow execution for multiple student teams without state collision or service degradation.	The architecture separates orchestration, parsing, reasoning, and persistence concerns. Project state is isolated per project identifier in Redis, and FastAPI asynchronous request handling supports concurrent pipeline operations.
Reliability	The system must continue operating safely when model confidence is low or input quality is	Low-confidence analysis triggers escalation and ambiguity resolution paths. Missing-data safeguards provide safe defaults and uncertainty flags. The GLM client

	incomplete, and it must fail gracefully when reasoning calls fail.	uses retry for transient HTTP failures, and agent execution returns structured error envelopes for controlled failure handling.
Maintainability	The codebase must remain modular, testable, and easy to evolve during and after the hackathon.	The backend is organized into clear modules including agents, orchestrator, parsers, edge-case handlers, schemas, and API routes. Unit and integration tests are provided for agent behavior, parser correctness, workflow flow, and route-level behavior.
Token Latency	Model-backed workflow stages should return within acceptable interactive latency for demonstration use, and long-running stages should still provide observable progress.	The system uses asynchronous API execution, bounded context slices in selected flows, and streaming endpoints to surface progressive stage output. The GLM client enforces request timeout behavior and supports controlled retries.
Cost Efficiency	The average model inference cost per project session should remain within practical hackathon limits while preserving output quality.	Prompting is stage-based and role-specific to avoid unnecessary monolithic prompts. Bounded text handling is used in selected paths, and only relevant context is passed per stage. Full caching and comprehensive context optimization are identified as planned enhancements.

3.3. Out of Scope / Future Enhancements

The following capabilities are part of the long-term product vision but are not included in the minimum viable product scope:

- A dedicated conflict resolution engine with decision matrix generation, structured voting workflows, and tie-breaking automation.
- A full semantic chunking and relevance-prioritized retrieval pipeline for large multi-source context handling.
- Mandatory user approval gates for all high-impact autonomous interventions before execution.

- Deep production integrations with learning management systems and institutional identity governance.
- Advanced analytics for multi-course and organization-wide benchmarking across teams.
- Native in-application communication channels with threaded decision support.
- Automated code quality and plagiarism intelligence integrated into readiness evaluation.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation

4.1.1. Grayscale Rollout and Comparative Testing Strategy

Our team will use a phased exposure strategy for post-hackathon deployment maturity. New orchestration features will first be enabled for a small subset of projects, followed by gradual expansion if reliability and quality metrics remain stable. Comparative testing can then be applied to evaluate alternative prompt or orchestration variants using measurable outcomes such as planning completeness, risk detection precision, and submission readiness accuracy.

4.1.2. Emergency Rollback (ER) & Golden Release

If severe regression is detected, our team will switch traffic to the most recent stable baseline release. Emergency rollback criteria include sustained service errors, repeated workflow stage failures, or model response instability that blocks core orchestration. During rollback, autonomous intervention features can be temporarily downgraded to advisory-only mode while preserving data integrity and user access to existing project state.

4.1.3. Priority Matrix

Priority	Trigger Condition	Action
Priority One Critical	Core workflow endpoints fail consistently, or model service outage prevents orchestration.	Trigger emergency rollback to stable baseline, switch to degraded advisory mode, and suspend autonomous stage execution until recovery is confirmed.
Priority Two High	Risk detection, planning, or readiness outputs become unreliable but service remains online.	Disable affected stage automation, route users through manual confirmation workflow, and deploy hotfix patch.
Priority Three Medium	Performance degradation, delayed streaming updates, or intermittent non-blocking errors.	Apply performance tuning, scale application resources, and monitor until service stabilizes.
Priority Four Low	Minor user interface defects or reporting inconsistencies with no execution impact.	Schedule in the next maintenance cycle and track through the standard backlog process.

4.2. Monitoring

The monitoring plan includes the following service health and quality indicators:

- Endpoint availability and error rates for all workflow routes.
- Model request latency, failure rate, and retry frequency.
- Workflow stage completion rate across analysis, planning, coordination, risk, and readiness stages.
- Redis and PostgreSQL connectivity, response time, and failure events.
- Frequency of low-confidence escalations and ambiguity resolution triggers.
- Deadline recovery and inactivity intervention trigger counts.

Alerting should be threshold-driven. If metrics exceed defined limits, incident severity should be classified using the priority matrix and corresponding response actions should be executed immediately.

4.3. Assumptions

- Users provide baseline project artifacts such as briefs, rubrics, or task context before orchestration starts.
- Team members have stable internet access during workflow interaction and updates.
- Activity events captured by the platform are sufficiently representative for participation analysis.
- External model service connectivity remains available during core orchestration requests.
- Deployment operates in a controlled environment with limited production-scale traffic.

4.4. External Dependencies

Tools	Purpose	Risks
Z.AI GLM API	Core reasoning engine for prompt inference, multi-step orchestration, and structured output generation.	High risk. Rate limiting, outages, or unstable responses can degrade or block core workflow intelligence. Mitigation includes retry handling, escalation paths, and advisory fallback behavior.
Redis	Stateful project memory, activity tracking, and decision log storage.	High risk. State continuity and orchestration context can be disrupted if unavailable. Mitigation includes health checks, reconnection strategy, and persistence safeguards.
PostgreSQL	Persistent storage for projects, tasks, documents, and related domain entities.	Medium risk. Data operations can fail during outage or high load. Mitigation includes connection pooling, migration discipline, and recovery procedures.

FastAPI Runtime & Hosting Environment	Service hosting for workflow endpoints, orchestration APIs, and streaming interfaces.	Medium risk. Runtime instability affects end-to-end availability. Mitigation includes rollback-ready deployments and application-level health monitoring.
Frontend Web Application Stack	User interaction layer for workflow operations and status visibility Medium risk. Client-side failures reduce usability even when the backend is healthy. Mitigation includes graceful error states and retry-capable request handling.	Medium risk. Client-side failures reduce usability even when the backend is healthy. Mitigation includes graceful error states and retry-capable request handling.

5. Project Management & Team Contribution

5.1. Project Timeline

The preliminary round started from 16th April to 26th April, and active development for CoordinaAI began on 21st April.

Day 1: Focused on project initiation and alignment. Our team finalized the problem definition, confirmed the minimum viable product scope, aligned the solution with hackathon judging requirements, and established the system architecture direction.

Days 2 to 4: Focused on development and integration. During this period, our team implemented GLM integration, agent reasoning logic, workflow orchestration, parser pipelines, shared project state, risk handling flows, and the user-facing dashboard experience. Our team also integrated backend and frontend components into a complete end-to-end demonstration flow.

Day 5 to 6: Focused on submission readiness. Our team conducted final validation to ensure consistency between implementation and documentation, fixed presentation-level issues, finalized the Product Requirements Document and

System Analysis Document, completed the pitch deck and video presentation for submission.

5.2. Team Members Role

Member	Role	Responsibilities	Key Deliverables
Jocelyn Gan Xin Yi	Platform and Experience Lead	Led user-facing platform experience, dashboard interfaces, interaction workflows, backend platform setup, data integration surfaces, activity visibility, and notification-oriented workflow presentation.	Functional demo interface, integrated workflow views, user interaction flows, and polished experience layer for evaluation.
Tan Yin June	Artificial Intelligence and Agent Intelligence Lead	Led GLM integration, prompt design, context handling, memory strategy, agent implementation, workflow reasoning logic, task decomposition, replanning behavior, and edge case reasoning paths.	Agent controller, reasoning pipelines, workflow decision logic, structured output endpoints, and core intelligence capabilities required by judging criteria.
Cheah Pui Yan	Documentation and Communications Lead	Led product documentation, system analysis documentation, consistency validation across documents and implementation, and pitch story development.	Product Requirements Document, System Analysis Document, architecture narrative consistency, and final pitch deck content.

5.3. Recommendations

- Introduce a formal context optimization layer that includes semantic chunking, relevance ranking, and controlled summarization for large multi-source project inputs.
- Add response caching for repeated reasoning contexts in analysis and readiness stages to reduce model cost and improve response time consistency.
- Implement stronger observability by tracking stage-level latency, model failure rates, confidence distributions, and escalation frequency through centralized metrics dashboards.
- Introduce explicit human confirmation gates for selected high-impact automated interventions to improve trust, governance, and fairness in team-sensitive actions.
- Expand deployment hardening with staged rollout, rollback checkpoints, and release health validation to reduce operational risk during future scaling.
- Strengthen participation analytics by combining activity events with richer contribution signals to reduce false inactivity classification and improve fairness.